Hafta 3: Araç İçi Haberleşme (CAN Bus Mesaj Kuyruğu)

(Struct, Dinamik Bellek Yönetimi, Pointer'lar ve Pair Programming)

Açıklama

Geçtiğimiz haftalarda aracımızın sürüş simülasyonunu başarıyla tamamladınız. Ancak gerçek bir elektromobilde, sensörler (hız, batarya, motor sıcaklığı vb.) ana bilgisayarla doğrudan konuşmazlar. Tüm bu modüller **CAN Bus** (Controller Area Network) adı verilen ortak bir iletişim hattına mesaj gönderirler.

Eğer ana bilgisayar (ECU) meşgulse ve birden fazla sensör aynı anda veri gönderirse ne olur? Bu mesajların kaybolmaması için bir bekleme sırasına girmeleri gerekir. İlk gelen mesaj, ilk işlenir. Bilgisayar bilimlerinde bu yapıya **Kuyruk (Queue) - FIFO (First In, First Out)** denir.

Bu hafta, 2-3 kişilik gruplar halinde çalışarak araç içi sensörlerden gelen mesajları toplayan ve sırayla işleyen bir CAN Bus Mesaj Kuyruğu simülasyonu yazacaksınız. Ancak bu sefer kuyruğumuzun boyutu kodun içinde sabit olmayacak, program başlarken **dinamik olarak (Dynamic Memory Allocation)** belirlenecek!

1. Versiyon Kontrolü: Pair Programming ve Kod İnceleme (Code Review)

Bu hafta kod yazmak kadar, bir takım olarak kod geliştirmeyi öğreneceksiniz. **Kendi yazdığınız kodu (Pull Request) kendiniz birleştiremezsiniz (Merge edemezsiniz).** Bunu takım arkadaşınız yapacak!

İzlemeniz Gereken Çiftli Çalışma (Pair Programming) İş Akışı:

1. **Hazırlık:** Takım üyelerinden biri GitHub'da yeni bir repository açar, aşağıdaki dosya yapısını (boş olarak) kurar ve diğer takım arkadaşını/arkadaşlarını projeye "Collaborator" olarak ekler.
2. **Kişi A'nın Görevi:** `feature/ana-menu` adında bir branch açar. `main.c` içindeki struct tanımlamalarını, dinamik bellek isteme ve menü döngüsünü tamamlar. Kodunu push'lar ve `main` dalına bir **Pull Request (PR)** açar.
3. **Kişi B'nin Görevi (Reviewer):** Kişi A'nın açtığı PR'ı inceler, kodda hata yoksa onaylar (Approve) ve `main` dalına **Merge** eder.
4. **Kişi B'nin Görevi (Geliştirici):** Kişi B kendi bilgisayarında `git pull` yaparak güncel `main` kodunu çeker. Ardından `feature/kuyruk-fonksiyonlari` adında yeni bir branch açar. `can_queue.c` içindeki dinamik bellek tahsisi (`malloc`), ekleme (Enqueue), çıkarma (Dequeue) ve yazdırma fonksiyonlarını yazar. Push'lar ve PR açar.
5. **Kişi A'nın Görevi (Reviewer):** Bu kez Kişi A, Kişi B'nin kodunu inceler, onaylar ve Merge eder.

2. Proje Dosya Altyapısı ve Teknik İsterler

Projenizi oluştururken karmaşayı önlemek adına aşağıdaki dosya hiyerarşisini ve isimlendirmeleri kullanmalısınız:

```
CAN_Bus_Simulasyonu/
│
├── can_queue.h      # Struct tanımları, include korumaları (#ifndef) ve fonksiyo
├── can_queue.c      # Kuyruk fonksiyonlarının (malloc, enqueue, dequeue, print)
└── main.c           # Kullanıcıdan kapasite alma, menü döngüsü ve bellek temizli
```

Dinamik Bellek Kullanımı: Dinamik bellek (`malloc` / `free` veya `calloc`) **KULLANILACAKTIR** ancak Linked List (Bağlı Liste) **KULLANILMAYACAKTIR**. Kuyruk, başlangıçta boyutu belirlenen *dinamik bir dizi (pointer)* ile yönetilecektir.

- `CAN_Mesaj` struct'ı içinde; mesaj ID'si (int), veri (float) ve gönderen sensör adı (char dizisi) bulunmalıdır.
- `MesajKuyrugu` struct'ı içinde; dinamik dizi pointer'ı (`CAN_Mesaj *mesajlar`), kapasite, front, rear ve count değerleri bulunmalıdır.

İpucu: Örnek Struct ve Queue Yapısı Aşağıdaki taslağı struct'larınızı oluştururken referans olarak alabilirsiniz:

```c
// 1. Tek bir sensör mesajının yapısı
typedef struct {
    int mesaj_id;           // Örn: 101 (Hız), 202 (Batarya)
    float veri;             // Örn: 45.5 (Derece veya km/s)
    char gonderen[30];      // Örn: "Motor_Sicaklik_Sensoru", burdaki array stati
                            // ve uzunluğunu istediğiniz gibi seçebilirsiniz
} CAN_Mesaj;

// 2. Mesajların beklediği kuyruk yapısı (Dinamik Dizi ile)
typedef struct {
    CAN_Mesaj *mesajlar;    // Dinamik olarak tahsis edilecek dizi pointer'ı
    int kapasite;           // Kullanıcının belirlediği maksimum kapasite
    int front;              // Kuyruğun başı (İşlenecek ilk mesajın indeksi)
    int rear;               // Kuyruğun sonu (Yeni eklenecek mesajın indeksi)
    int count;              // Kuyruktaki anlık mesaj sayısı
} MesajKuyrugu;
```

3. Görevler ve İş Mantığı (Ana Menü)

Uygulamanız başladığında ilk olarak kullanıcıdan kuyruk kapasitesini istemeli, belleği ayırmalı ve ardından ana menüyü sunmalıdır:

[BAŞLANGIÇ] CAN Bus Kapasitesini Belirleyin (Örn: 5, 10, 50): Ardından döngüye girer:

1. Sensörden Mesaj Üret (Enqueue):

- Kullanıcıdan mesaj ID'si, veri değeri ve sensör adını girmesini ister.
- Bu bilgileri bir `CAN_Mesaj` struct'ı haline getirip kuyruğa ekler.

2. Bekleyen İlk Mesajı İşle (Dequeue):

- Kuyrukta mesaj varsa, `front` sırasındaki mesajı kuyruktan çıkarır.
- Mesajın içeriğini ekrana yazdırır: *"[İŞLENDİ] Motor_Sicaklik_Sensoru (ID: 101) -> Veri: 55.0"*

3. Bekleyen Mesajları Göster (Print Queue):

- Kuyruktaki anlık `count` sayısını ekrana yazdırır.
- **ÖNEMLİ:** Kuyruktaki mesajları sadece sıradan bir dizi gibi 0'dan kapasiteye kadar yazdırmak yerine; işlenecek ilk elemandan (`front`) başlayarak sırayla en son eklenen elemana (`rear`) kadar dolaşan bir döngü kurmalısınız.

4. Sistemi Kapat (Memory Free):

- Programdan çıkmadan önce dinamik olarak ayırdığınız belleği RAM'e iade etmeniz (`free`) zorunludur!

4. Sınır Durumlar (Edge Cases) ve Hata Yönetimi

Veri yapıları tasarlarken en önemli şey sınırları kontrol etmektir:

- **Kuyruk Taşması (Queue Overflow):** Eğer kuyrukta kapasite kadar mesaj biriktiyse, yeni bir mesaj eklenmeye çalışıldığında hata vermelidir: `"[HATA]: CAN Bus hattı dolu! Yeni mesaj eklenemez."`
- **Boş Kuyruk (Queue Underflow):** Eğer kuyrukta hiç mesaj yoksa (`count == 0`) ve mesaj işlenmeye/yazdırılmaya çalışılırsa uyarı verilmelidir: `"[BİLGİ]: İşlenecek yeni mesaj bulunmuyor."`
- **Memory Leak (Bellek Sızıntısı):** `malloc` ile ayırdığınız belleği, programın sonunda `free` ile temizlemezseniz bilgisayarın RAM'inde yer işgal etmeye devam eder. Gömülü sistemlerde bu durum kilitlenmelere yol açar.

5. Raporlama Kuralları

Önceki ödevde olduğu gibi, bu projenin de bir teknik raporu olacaktır. Dosyalarınızın arasına `Rapor.pdf` (veya docx) eklemeyi unutmayın.

- **Format:** "Bölüm 9: Rapor Yazım Kılavuzu" kuralları geçerlidir (Arial, 12 punto, 1.15 satır aralığı, %20 benzerlik sınırı).
- **İçerik:** İkinci ödevde olduğu gibi, öncelikle yazdığınız kodun (özellikle `can_queue.c` içindeki dinamik bellek yönetimi ve kuyruk fonksiyonlarının) nasıl çalıştığını adım adım anlatmalısınız. Ayrıca raporunuzda şu sorulara da cevap vermelisiniz:
    1. Neden sabit boyutlu bir dizi yerine `malloc` ile dinamik bellek ayırmayı tercih ettik? Bunun araca / sisteme sağladığı avantaj nedir?

2. **Stack (Yığın) Ön Bilgisi:** Bilgisayar bilimlerinde "Stack" yapısı, son giren verinin ilk çıktığı (LIFO - Last In, First Out) bir mantıkla çalışır (örn: üst üste dizilmiş bulaşık tabakları veya bilgisayardaki CTRL+Z geri alma işlemi). Peki, araç içi haberleşmede (CAN Bus) mesajların işlenmesi için neden LIFO (Stack) yerine ilk girenin ilk çıktığı FIFO (Queue - Kuyruk) veri yapısını tercih ettik?
3. Takım arkadaşınızın kodunu incelerken (Code Review) nelere dikkat ettiniz?

Örnek Çıktı (Kendinize göre değiştirebilirsiniz)

```
[SİSTEM] Lütfen CAN Bus mesaj kuyruğu kapasitesini giriniz: 2
[BİLGİ] 2 kapasiteli dinamik mesaj kuyruğu bellekte oluşturuldu!

--- CAN BUS İLETİŞİM SİSTEMİ ---
1. Sensörden Mesaj Üret (Kuyruğa Ekle)
2. Bekleyen İlk Mesajı İşle (Kuyruktan Çıkar)
3. Bekleyen Mesajları Göster
4. Çıkış
Seçiminiz: 1

Sensör Adı: Batarya_Sicaklik
Mesaj ID: 201
Veri: 45.5
[BİLGİ] Mesaj kuyruğa eklendi. (Sıradaki mesaj sayısı: 1)

...
Seçiminiz: 1
Sensör Adı: Tekerlek_Hiz
Mesaj ID: 105
Veri: 60.2
[BİLGİ] Mesaj kuyruğa eklendi. (Sıradaki mesaj sayısı: 2)

...
Seçiminiz: 1
Sensör Adı: Fren_Pedali
Mesaj ID: 301
Veri: 1.0
[HATA] CAN Bus hattı dolu! Kapasite (2) aşıldı. Yeni mesaj eklenemez.

...
Seçiminiz: 3
[BEKLEYEN MESAJLAR - Toplam: 2]
1. Batarya_Sicaklik (ID: 201) - Veri: 45.50
2. Tekerlek_Hiz (ID: 105) - Veri: 60.20

...
Seçiminiz: 2
[İŞLENDİ] Batarya_Sicaklik (ID: 201) sensöründen gelen '45.50' verisi ana bilgisc
[BİLGİ] Kuyrukta kalan mesaj sayısı: 1

...
Seçiminiz: 4
[BİLGİ] Dinamik bellek (RAM) temizleniyor...
[BİLGİ] Sistem güvenli bir şekilde kapatıldı.
```